

Midterm review

Hyunjong Choi, Ph.D.
Assistant Professor of Computer Science
San Diego State University



Midterm #1

☐ Date

- ✓ 9/25 (Thursday) during the class

☐ Closed book

- ✓ One cheat paper is allowed (for both sides)
- ✓ You can make the cheat paper either printed or by handwritten.
- ✓ But, NO calculator, NO electric devices

☐ 5 questions, each has subproblems

- ✓ Covers all topics from lecture slides
- ✓ Complete the pseudocode
- ✓ Understand the divide-and-conquer approach
- ✓ Find the solutions from a recurrence equation
 - Exact solutions
 - Asymptotic solutions

Example 1

□ For the given Algorithm function,

- 1) Determine the total number of iterations executed by the loop structure, i.e., the number of iterations of L4.
- 2) Compute the final value of s after Algorithm completes.

Algorithm Loop (n):

L1. $s = 0$

L2. For $i = 1$ to n^2 do

L3. For $j = 1$ to i do

L4. $s = s + i$

1) # of iterations

$$\sum_{i=1}^{n^2} i = \frac{n^2(n^2 + 1)}{2} = \frac{n^4 + n^2}{2}$$

2) S value

$$\begin{aligned} \sum_{i=1}^{n^2} i^2 &= \frac{n^2(n^2 + 1)(2n^2 + 1)}{6} \\ &= \frac{2n^6 + 3n^4 + n^2}{6} \end{aligned}$$

Example 2 – 1)

□ For the given recurrence equation.

$$T(n) = \begin{cases} 1 & , \text{if } n = 1 \\ 2T\left(\frac{n}{2}\right) + n & , \text{if } n > 1 \end{cases}$$

1) What is asymptotic solution of $T(n)$?

If we apply master theorem,
By case 2, $T(n) = \Theta(n \cdot \lg(n))$

Example 2 – 2)

2) Find an exact solution for $T(n)$ when n is a power of 2. (Try to solve it by math without using a recursion tree).

$$\begin{aligned} T(n) &= 2T\left(\frac{n}{2}\right) + n && \leftarrow T\left(\frac{n}{2}\right) = 2T\left(\frac{n}{4}\right) + \frac{n}{2} \\ &= 2\left\{2T\left(\frac{n}{4}\right) + \frac{n}{2}\right\} + n && \leftarrow T\left(\frac{n}{4}\right) = 2T\left(\frac{n}{8}\right) + \frac{n}{4} \\ &= 2^2T\left(\frac{n}{4}\right) + 2n \\ &= 2^2\left\{2T\left(\frac{n}{8}\right) + \frac{n}{4}\right\} + 2n \\ &= 2^3T\left(\frac{n}{8}\right) + 3n \\ &\dots && \leftarrow n = 2^k \\ &= 2^kT\left(\frac{n}{2^k}\right) + k \cdot n && \leftarrow k = \lg n \\ &= k \cdot n + 2^kT(1) = kn + 2^k \\ &= n \lg n + n = n(\lg n + 1) \end{aligned}$$

Example 2 – 3)

3) Demonstrate the correctness of your result of 2) by showing $T(16)$

*Hint: find the value of $T(16)$ using recurrence equation, then check with the answer from 2).

$$\begin{array}{l} T(16) = 2 \cdot T(8) + 16, \quad T(8) = 2 \cdot T(4) + 8, \quad T(n) = n \cdot (\log_2 n + 1) \\ = 2 \cdot \{2 \cdot T(2) + 4\} + 16, \quad T(4) = 2 \cdot T(2) + 4 \\ = 2^2 \cdot T(2) + 32 \\ = 2^2 \cdot \{2 \cdot T(1) + 2\} + 32 \\ = 2^3 \cdot T(1) + 48, \quad T(2) = 2 \cdot T(1) + 2 \\ = 2^3 \cdot 4 + 48 \\ = 80. \end{array}$$

$\downarrow$
At $n=16$,
 $T(16) = 16 \cdot (\log_2 16 + 1)$
 $= 16 \cdot (4 + 1)$
 $= 80$

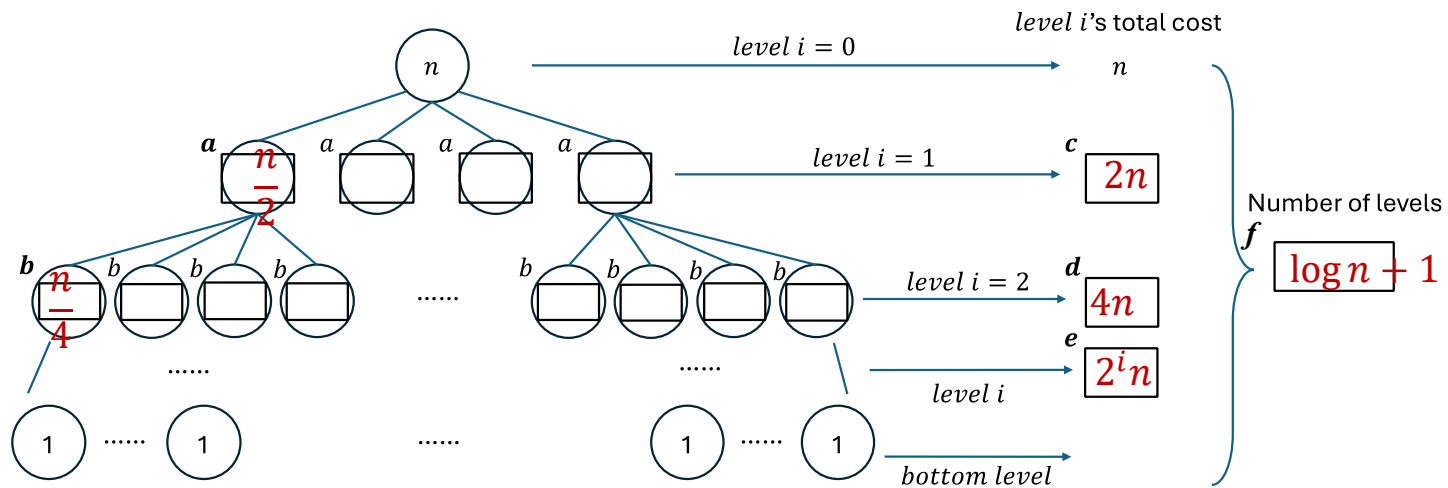
Our exact solution of 2) is correct

Example 3 – 1)

□ For the given recurrence equation.

$$T(n) = \begin{cases} 1 & , \text{if } n = 0 \\ 4T\left(\frac{n}{2}\right) + n & , \text{if } n > 1 \end{cases}$$

1) Fill out the blanks (a to g) while sketching the recursion tree of the above recurrence.



Example 3 – 2)

2) Now, find the exact solution of the above recurrence based on the results of 1).

*Hint: You may use the following geometric series:

$$\sum_{k=0}^n r^k = \frac{1 - r^{n+1}}{1 - r}$$

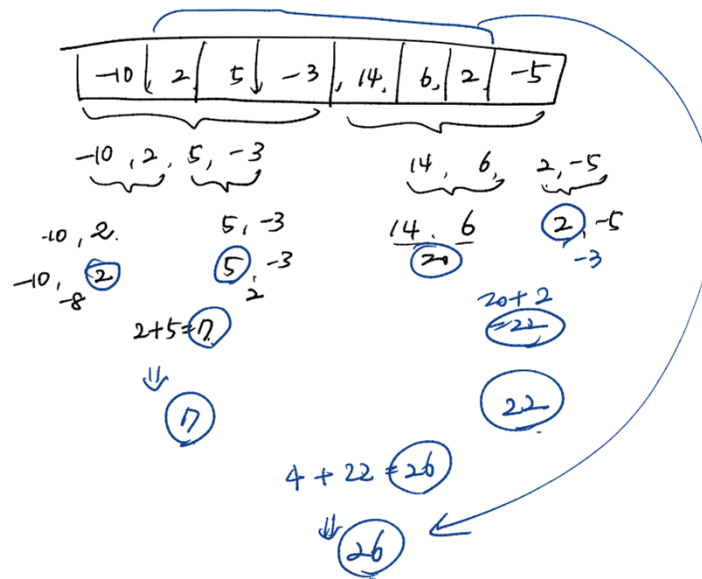
$$\begin{aligned} & \sum_{i=0}^{\log n} 4^i \cdot \frac{n}{2^i} \\ &= n \cdot \sum_{i=0}^{\log n} 2^i = n \cdot \frac{2^{\log n + 1} - 1}{2 - 1} \\ &= n \cdot \{2^{\log n + 1} - 1\} = n \cdot (2 \cdot n - 1) \\ &= \underline{2n^2 - n} \end{aligned}$$

Example 4 – 1)

□ For the given array A,

$$A = [-10, 2, 5, -3, 14, 6, 2, -5]$$

1) Find the maximum sum of any contiguous subarray in array A. (Justify your answer)



Example 4 – 2)

- ✓ Find that contiguous subarray of A that results the maximum sum, i.e., your answer (subarray) should give the maximum sum of the above question.

[2,5,-3,14,6,2]

Example 5

- Complete the following recursive algorithm FIND_MIN_MAX, employing a divide-and-conquer approach, to find both the minimum and the maximum elements in an array (A) of elements. This algorithm should return a pair (a, b), where a is the minimum element and b is the maximum. What is the running time of your method?

*Hint: You can use min(a, b) and max(a, b) built-in function where each returns the minimum and the maximum number for two numbers with efficiency of $\Theta(1)$.

1) Complete the following Pseudocode (on the next page).

Example 5 – 1)

- ✓ start and end are indices of input array A
- ✓ value_min and value_max are the minimum and maximum values

```
FIND_MIN_MAX(A, start, end) {  
    if start == end  
        // Case: array A has only one element.  
        return (A[start], A[end])  
    else if end - start == 1  
        if A[start] > A[end]  
            return (A[end], A[start])  
        else  
            return (A[start], A[end])  
    else  
        // Divide step  
        mid = [start + end]/2  
        // Conquer step  
        (min1, max1)=FIND_MIN_MAX(A, start, mid)  
        (min2, max2)=FIND_MIN_MAX(A, mid+1, end)  
        // Combine step code  
        value_min = min(min1, min2)  
        value_max = max(max1, max2)  
        return (value_min, value_max)  
}
```

Example 5 – 2)

- What is the complexity of FIND_MIN_MAX algorithm? (Use asymptotic notation Θ)

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(1)$$

by master theorem, $T(n) = \Theta(n)$

Example 6

□ Use mathematical induction to show that the solution of the recurrence

$$T(n) = \begin{cases} 2 & \text{if } n = 2, \\ 2T\left(\frac{n}{2}\right) + n & \text{if } n = 2^k, \text{ for } k > 1 \end{cases}$$

is $T(n) = n \lg(n)$.

*Hint: Assume that n is an exact power of 2, i.e., $n = 2^k$

1. Base case :

For $n=2^1$, $T(n) = 2 \lg 2 = 2$.

2. Inductive step :

Suppose $n=2^k$, $T(n) = n \cdot \lg(n) = 2^k \cdot \lg(2^k) = 2^k \cdot k$

for $n=2^{k+1}$,

$$\begin{aligned} T(n) &= 2T\left(\frac{2^{k+1}}{2}\right) + 2^{k+1} \\ &= 2 \cdot T(2^k) + 2^{k+1} \\ &= 2 \cdot 2^k \cdot k + 2^{k+1} \\ &= 2^{k+1} \cdot k + 2^{k+1} \\ &= 2^{k+1} \cdot (k+1) \quad \leftarrow \text{by } \lg n = k+1 \\ &= 2^{k+1} \cdot \lg n \quad \leftarrow \text{by } n = 2^{k+1} \\ &= 2^{k+1} \cdot \lg 2^{k+1} \\ &\Rightarrow \underline{\underline{n \cdot \lg n}} \quad \text{True.} \end{aligned}$$

Example 7

□ Solve the following recurrence equation by applying master theorem.

$$T(n) = 3T\left(\frac{n}{2}\right) + n^2$$

$$n^{\log_2 3} = n^{1.58}, \quad f(n) = n^2$$

✓ $f(n) = n^2 \geq c \cdot n^{1.58+\epsilon}$, satisfied as long as $\epsilon < 0.42$

✓ regularity condition: $af\left(\frac{n}{b}\right) \leq c \cdot f(n)$

$$\Rightarrow 3 \cdot \left(\frac{n}{2}\right)^2 \leq c \cdot n^2$$

$$= \frac{3}{4}n^2 \leq c \cdot n^2 \quad \text{hold if } \underline{c = \frac{3}{4} < 1}$$

∴ apply case 3

$$\underline{T(n) = \Theta(n^2)}$$

Example 8

□ Solve the following recurrence equation by applying master theorem.

$$T(n) = 2^n \times T\left(\frac{n}{2}\right) + n^n$$

⇒ We cannot apply master theorem, b.c. 2^n is not a constant.

Example 9

□ Solve the following recurrence equation by applying master theorem.

$$T(n) = \sqrt{2} \times T\left(\frac{n}{2}\right) + \log(n)$$

$$n^{\log_2 \sqrt{2}} = n^{\frac{1}{2}} = \sqrt{n}, \quad f(n) = \log n.$$

$\log(n) \leq C \cdot n^{\frac{1}{2} - \epsilon}$, we can find $\epsilon > 0$ b.c. polynomial function is larger than logarithm function.

$\therefore$ apply case I. $\therefore T(n) = \underline{\Theta(n^{\frac{1}{2}})}$

Example 10

□ Solve the following recurrence equation by applying master theorem.

$$T(n) = 0.5 \times T\left(\frac{n}{2}\right) + \frac{1}{n}$$

⇒ We cannot apply master theorem b.c. $a < 1$.

Example 11

□ Solve the following recurrence equation by applying master theorem.

$$T(n) = 3 \times T\left(\frac{n}{3}\right) + \frac{n}{2}$$

$$n^{\log_3 3} = n, \quad f(n) = \frac{n}{2}.$$

$$\Rightarrow \text{apply case 2,} \quad \therefore T(n) = \underline{\Theta(n \log n)}.$$